Universidad Nacional de Río Cuarto

Facultad de Ciencias Exactas, Físico-Químicas y Naturales
Departamento de Computación

CDCS Mini

Una herramienta de análisis de contratos comportamentales
embebidos en código Python

Proyecto de tesis de la carrera
Carrera: Licenciatura en Ciencias de la Computación

Tomás Rodeghiero

tomyrodeghiero@gmail.com

Desafío técnico propuesto por Daniel Gutson

14 de mayo de 2026

Índice

Resumen **2**

Abstract **2**

1. Introducción **3**
1.1. Contexto y motivación . . . 3
1.2. Problema . . . 3
1.3. Aporte . . . 3

2. La idea de CDCS Mini **4**
2.1. Qué es un contrato `@generate` . . . 4
2.2. Salida esperada . . . 4

3. Arquitectura **5**

4. El núcleo de análisis **5**
4.1. Por qué AST y no expresiones regulares . . . 5
4.2. Parser del DSL . . . 6
4.3. Validaciones . . . 6
4.4. Reporte determinístico . . . 6

5. Capa CLI **6**

6. Capa HTTP: FastAPI **7**

7. Frontend: Next.js + Tailwind **7**

8. Criterios de calidad **7**

9. Organización del código **8**

10. Por qué sirve como base para una tesina **8**

11. Posibles líneas futuras **9**
11.1. Especificación formal del DSL . . . 9
11.2. Sistema de tipos sobre los contratos . . . 9
11.3. Generación de tests por propiedades . . . 9
11.4. Integración como extensión de editor . . . 9
11.5. Soporte para métodos y clases . . . 9

12. Conclusiones **9**

Referencias **10**

Resumen

CDCS Mini es una herramienta de análisis estático que recibe un archivo Python como entrada y produce un reporte JSON determinístico a partir de *contratos comportamentales* escritos en docstrings con la marca `@generate`. La herramienta no ejecuta el código analizado: lo parsea con el módulo `ast` de la biblioteca estándar, interpreta un pequeño DSL embebido (con secciones `behavior`, `examples` y `constraints`), corre validaciones semánticas, y emite el reporte estructurado. La salida está pensada para ser estable entre ejecuciones, lo que hace que el sistema sea verificable y útil como insumo de otras herramientas.

El proyecto incluye cuatro piezas: un núcleo en Python con tipado estricto, un CLI (`cdcs-mini`), un backend HTTP construido con FastAPI que reutiliza la misma lógica, y un frontend en React + Next.js + Tailwind que consume el backend. El presente documento describe el problema que motiva la herramienta, las decisiones de diseño tomadas, el flujo general del análisis, los criterios de calidad aplicados y por qué este proyecto puede servir como base para una tesina de Licenciatura en Ciencias de la Computación.

Palabras clave: análisis estático, AST, DSL, lenguajes específicos de dominio, herramientas tipo compilador, contratos comportamentales.

Abstract

CDCS Mini is a static analysis tool that consumes a Python source file and produces a deterministic JSON report from *behavioral contracts* embedded in docstrings via an `@generate` marker. The tool never executes the analyzed code: it parses it through the standard `ast` module, interprets a small DSL with `behavior`, `examples` and `constraints` sections, runs a fixed pipeline of semantic validators, and emits a structured, stable report.

The project ships four artefacts: a strictly typed Python core, a CLI (`cdcs-mini`), a FastAPI HTTP layer that reuses the same logic, and a Next.js + Tailwind frontend that consumes the API. This document describes the problem behind the tool, the design decisions made, the overall analysis flow, the quality criteria applied, and why the system is a reasonable starting point for an undergraduate thesis in Computer Science.

Keywords: static analysis, AST, DSL, domain-specific languages, compiler-like tooling, behavioral contracts.

1. Introducción

1.1. Contexto y motivación

En los últimos años se popularizó la idea de *programar describiendo intenciones*: dejar especificaciones legibles cerca del código (en docstrings, en archivos auxiliares, en anotaciones) y hacer que una herramienta externa las consuma, verifique o derive comportamiento a partir de ellas. Cuando la especificación está embebida en el propio archivo fuente, surge un problema clásico de *front-end de compilador*: hay que parsear, interpretar y validar un fragmento de un lenguaje no trivial sin recurrir a heurísticas frágiles.

Este trabajo se ubica en ese cruce: tomar un archivo Python, descubrir las funciones que llevan un contrato `@generate`, parsear el contrato como una DSL pequeña pero estructurada, validar que sea coherente con la firma de la función, y emitir un reporte JSON. El énfasis no está en *generar* código a partir del contrato, sino en construir las primeras fases de la tubería de procesamiento que cualquier herramienta posterior necesitaría como base sólida.

1.2. Problema

Verificar contratos como los del ejemplo de la figura 1 obliga a resolver varios sub-problemas a la vez:

- localizar las funciones marcadas con `@generate` sin recurrir a expresiones regulares sobre el archivo Python;
- recuperar de cada función su firma exacta (parámetros, anotaciones, tipo de retorno);
- parsear el cuerpo del contrato como una DSL multilineal con secciones;
- detectar que cada referencia a una variable dentro del contrato corresponde a un parámetro real de la función;
- rechazar firmas no soportadas (por ejemplo, variádicas) y contratos sin ejemplos;
- emitir todo lo anterior en un formato JSON que sea *determinístico*: misma entrada $\Rightarrow$ misma salida, byte a byte.

Si bien cada uno de esos problemas es modesto por separado, su composición exige una arquitectura ordenada. El proyecto se propone resolverlos con el menor acoplamiento posible y con un código que se sostenga frente a una revisión exigente.

1.3. Aporte

CDCS Mini aporta una implementación de referencia, en Python 3.12, de un analizador de contratos `@generate` junto con dos superficies de consumo (CLI y HTTP) y un cliente web. El énfasis está en la separación de responsabilidades, el tipado estricto, la cobertura mediante tests automatizados y la trazabilidad del análisis a través de diagnósticos estilo *compiler-style*.

Figura 1: Ejemplo canónico de un contrato `@generate`.

```
def parse_port(value: str) -> int:
    """@generate
    behavior:
      strip(value)
      require value matches digits
      require 1 <= int(value) <= 65535
      return int(value)

    examples:
      parse_port("80") == 80
      parse_port("443") == 443
      parse_port("0") raises ValueError

    constraints:
      no_imports
      no_network
      no_filesystem
    """
    ...
```

2. La idea de CDCS Mini

2.1. Qué es un contrato `@generate`

Un contrato `@generate` es un docstring estructurado que describe el comportamiento esperado de una función. Está pensado para ser legible por una persona y por una máquina al mismo tiempo. El docstring siempre empieza con el marcador `@generate` y a continuación trae hasta tres secciones, todas opcionales, en orden libre:

- `behavior:` una secuencia de pasos. Cada línea es una operación (`strip(value)`), una precondición (`require ...`) o un retorno (`return ...`).
- `examples:` pares entrada/salida o pares entrada/excepción, expresados con `==` o `raises`.
- `constraints:` declaraciones cualitativas (`no_imports`, `no_network`, etc.).

2.2. Salida esperada

Para el ejemplo de la figura 1, la salida es:

```
{
  "functions": [
    {
      "name": "parse_port",
      "status": "ok",
      "parameters": { "value": "str" },
      "returns": "int",
      "examples": 3,
      "constraints": ["no_imports", "no_network", "no_filesystem"]
    }
  ],
  "errors": []
}
```

El formato sigue la especificación original del desafío. Mantenerlo intacto es una decisión deliberada: el reporte es un *contrato de salida* que otras herramientas (o tests de regresión) pueden consumir asumiendo estabilidad estructural.

3. Arquitectura

El sistema se estructura en capas concéntricas. Hacia adentro está la lógica que define *qué* hace la herramienta; hacia afuera están las superficies que la exponen al usuario.

1. **Dominio**. Modelos inmutables (`dataclass` con `frozen=True, slots=True`): `Parameter`, `Signature`, `BehaviorStep`, `Example`, `Contract`, `FunctionReport`, `Report`, `Diagnostic`. No tienen lógica, solo forma.
2. **Parsing**. Dos parsers: `SourceParser` sobre `ast` de Python y `DSLParser` sobre el cuerpo del docstring.
3. **Validación**. Validators puros que reciben un `Signature` y un `Contract` y devuelven una lista de `Diagnostic`.
4. **Reporte**. Un `JsonReporter` serializa el `Report` al esquema `ReportDict` (un `TypedDict` compartido con el backend).
5. **Aplicación**. Un `ReportService` orquesta las capas anteriores en un orden fijo.
6. **Superficies**. CLI (`cdcs-mini`), backend HTTP (FastAPI) y frontend (Next.js).

El flujo general es lineal: `source` → `ParsedSource` → `Contract` → diagnósticos → `Report` → JSON.

4. El núcleo de análisis

4.1. Por qué AST y no expresiones regulares

La especificación del desafío es explícita: no se permite parsing del código Python con expresiones regulares. Más allá de la regla, hay razones técnicas claras para usar el módulo `ast` de la biblioteca estándar:

- el *lexer/parser* de Python ya está implementado, probado y soporta versiones del lenguaje futuras;
- el AST trae los números de línea y columnas exactas para diagnósticos;
- se distinguen sin ambigüedad funciones síncronas y asíncronas, parámetros posicionales/keyword-only, variádicos, anotaciones, retornos;
- `ast.unparse` permite reconstruir representaciones canónicas de las anotaciones (`int`, `list[str]`, etc.) sin reinventar el formateo.

El parser de fuentes recorre el `Module` y extrae las funciones de nivel superior (los métodos y funciones anidadas quedan fuera del alcance del contrato). De cada una toma la firma y, si existe, el docstring que empieza con `@generate`.

4.2. Parser del DSL

El cuerpo del docstring se parsea con una máquina de estados muy simple, línea a línea. Solo se identifican tres encabezados (`behavior:`, `examples:`, `constraints:`); cualquier otro encabezado dispara un diagnóstico `MalformedDSLError`. Dentro de `behavior` cada línea se clasifica en tres formas:

- `require X matches Y`: un *matcher* estructural donde `X` es el sujeto y `Y` el nombre del matcher (`digits`, `alpha`, ...);
- `require <expresión Python>`: la expresión se re-parsea con `ast.parse(..., mode=éval')` para extraer los identificadores;
- `return <expresión>` o llamadas tipo `strip(value)`.

Cuando la línea es interpretada como Python, las únicas identidades que se recogen como referencias a parámetros son las que *no* aparecen como `func` de una llamada. Es decir, en `int(value)` se extrae solo `value`; `int` es una operación. Este criterio mantiene la señal de diagnóstico limpia: solo se reportan parámetros realmente desconocidos.

4.3. Validaciones

Los validators son funciones puras tipadas con un `Protocol`:

```
class ContractValidator(Protocol):
    def __call__(
        self, *, signature: Signature, contract: Contract, function_line
            : int
    ) -> Iterable[Diagnostic]: ...
```

Hoy hay dos validators: `validate_examples_present` (emite `MissingSamplesError` si la sección `examples` no existe o está vacía) y `validate_known_parameters` (emite `InconsistentPromptError` si alguna referencia del `behavior` no corresponde a un parámetro ni a un builtin de Python). Esta lista se puede ampliar sin tocar el resto del sistema.

4.4. Reporte determinístico

El esquema de salida está formalizado como un `TypedDict` (`ReportDict`) que vive en el módulo `cdcs_mini.reporting.schema`. El `JsonReporter` lo serializa con `json.dumps(..., sort_keys=False, ensure_ascii=False)`: el orden de las claves lo controla el código, no el encoder. Los diagnósticos se ordenan antes de serializar por la tupla `(line, code, message)`; la `dataclass` `Diagnostic` se declara con `order=True` para que ese ordenamiento sea derivable automáticamente. El resultado es que *misma entrada produce misma salida byte a byte*, una propiedad que se verifica con un test dedicado.

5. Capa CLI

El CLI se invoca como pide la especificación:

```
cdcs-mini input.py --out report.json
```

Acepta también imprimir a `stdout` si se omite `-out`. Los códigos de salida son:

- `0`: el reporte se generó sin diagnósticos;
- `1`: el reporte se generó pero con uno o más diagnósticos (el archivo se escribe igual);
- `2`: error de uso (archivo inexistente, argumentos inválidos).

Los diagnósticos se imprimen por `stderr` en un formato compacto estilo compilador (`Code: line N: message in function`). El propio CLI es muy delgado: parsea argumentos, lee el archivo, invoca al `ReportService`, escribe la salida y decide el código de retorno.

6. Capa HTTP: FastAPI

El backend expone tres endpoints:

- `GET /health`: liveness probe;
- `POST /reports/from-source`: recibe `{filename, source}`;
- `POST /reports/from-file`: recibe un `.py` como upload.

Lo importante de esta capa es que *no* duplica lógica del CLI. Ambos endpoints construyen una respuesta usando el mismo `ReportService.default()` y el mismo `JsonReporter`, declarados como singletons a nivel de módulo. Como el `ReportDict` ya es un `TypedDict` compartido, el modelo de respuesta de pydantic (`ReportResponse`) lo referencia directamente: la documentación OpenAPI describe la salida con precisión campo por campo, sin manualidades. La documentación interactiva queda accesible en `/docs` (Swagger UI) y `/redoc`.

7. Frontend: Next.js + Tailwind

El frontend es una aplicación Next.js con TypeScript estricto y Tailwind. No replica nada de la lógica del parser: solo permite subir un `.py` o pegar código, hace `POST` al backend y renderiza el reporte. Incluye un visor JSON con copia al portapapeles, una grilla de métricas (funciones encontradas, diagnósticos, constraints), una lista compacta de diagnósticos cuando los hay y un identificador determinístico de reporte calculado como SHA-256 del JSON.

La interfaz soporta modo claro y oscuro. El modo oscuro se aplica antes de la hidratación de React mediante un script inline en el `<head>` que consulta `localStorage` y `prefers-color-scheme` para evitar el destello inicial. La paleta se diseñó para verse cuidada en ambos temas, con un acento índigo/violeta consistente.

8. Criterios de calidad

El proyecto se sometió a una serie de criterios objetivos:

1. **Tipado estricto.** Todo el código Python pasa `mypy-strict`. Solo aparece `Any` en docstrings de prosa: las anotaciones reales usan `TypedDict`, `Literal`, `Protocol` y `frozenset[str]` donde corresponde.
2. **Determinismo verificable.** Hay un test que ejecuta la generación dos veces sobre la misma entrada y compara los strings de salida.
3. **Sin `eval` ni `exec`.** El parser no evalúa expresiones; solo las parsea con `ast.parse(mode=éval')` para extraer identificadores.

4. **Tests.** 35 tests organizados por capa (parser de fuentes, parser DSL, validators, servicio, CLI, backend), con *fixtures* reales que cubren los caminos felices y los varios tipos de error.
5. **Lint.** `ruff` en estricto, con reglas de `pyflakes`, `pyupgrade`, `bugbear` y `isort`.
6. **Inmutabilidad.** Los modelos del dominio son `frozen=True, slots=True`; ningún paso de la pipeline muta valores.
7. **Frontend tipado.** `tsc -noEmit` limpio.

9. Organización del código

```
cdcs-mini/
  pyproject.toml
  src/cdcs_mini/
    cli.py
    domain/{models,diagnostics}.py
    parsing/{source_parser,dsl_parser}.py
    validation/validators.py
    reporting/{schema,json_reporter}.py
    application/report_service.py
  backend/app/{main,schemas}.py
  frontend/{app,components,lib}/...
  tests/{fixtures,test_*.py}
```

Cada paquete tiene una responsabilidad acotada y conoce únicamente a los inmediatamente anteriores: `parsing` no conoce `validation`; `validation` no conoce `reporting`; `reporting` no conoce a `parsing`. `application` es el único punto donde las capas se encuentran. Esta estructura facilita los tests por unidad y permite reemplazar piezas (por ejemplo, agregar un validator o un formato de salida) sin tocar el resto.

10. Por qué sirve como base para una tesina

El recorte temático de *CDCS Mini* toca, en escala pequeña, varias áreas centrales de la carrera de Licenciatura en Ciencias de la Computación:

- **Lenguajes y compiladores**. La herramienta implementa, de manera honesta, las primeras fases de un compilador: análisis léxico, sintáctico y semántico, más la emisión de un IR (en este caso, el JSON). Cada uno de estos pasos puede profundizarse: gramática formal para el DSL, sistemas de tipos sobre los contratos, optimizaciones del IR, etc.
- **Métodos formales**. Los contratos `@generate` son una forma incipiente de especificación. Una tesina podría extender el DSL hacia algo más cercano a *Design by Contract* y derivar tests automáticos o verificaciones por propiedades.
- **Ingeniería de software**. El proyecto pone en práctica patrones reconocibles (parser combinator implícito en el DSL, repository/service en el orquestador, builder en el reporter) y políticas de calidad medibles (`mypy -strict`, tests, lint).
- **Sistemas y arquitectura**. La separación CLI/HTTP/Web sobre un mismo núcleo es un caso pequeño pero completo del patrón *hexagonal* o *ports and adapters*.

A diferencia de proyectos de tesina más amplios, este tiene un alcance acotado pero defendible: un sistema completo, andando, con código limpio, decisiones explicables y resultados observables.

Eso lo vuelve un buen punto de partida para profundizar en una dirección concreta sin gastar el primer semestre construyendo la infraestructura.

11. Posibles líneas futuras

11.1. Especificación formal del DSL

Actualmente el DSL se describe en el README y en los tests. Una línea natural es escribir una gramática EBNF, derivar de allí un parser por descenso recursivo o usar una herramienta como *Lark* y demostrar equivalencia con la implementación actual. Esto da un marco para razonar sobre extensiones (por ejemplo, condicionales en `behavior`) sin romper la implementación.

11.2. Sistema de tipos sobre los contratos

Los `require` hoy son chequeos sintácticos. Una extensión interesante es darles semántica: tratar a cada `require` como una pre-condición y a cada `return` como una post-condición, y derivar un sistema de tipos por refinamiento sencillo (*e.g.*, `int` restringido al rango `[1, 65535]`). Eso conectaría el trabajo con literatura sobre *refinement types*.

11.3. Generación de tests por propiedades

Otra dirección es leer el contrato y emitir un test de `Hypothesis`: un *strategy* para cada parámetro a partir de su anotación, los predicados de `require` como filtros, y los `examples` como casos concretos. La meta no es generar el código de la función, sino la batería de tests que la función debería pasar.

11.4. Integración como extensión de editor

Empaquetar al núcleo como un Language Server (LSP) permitiría tener diagnósticos en vivo dentro de VS Code o Neovim mientras se escribe el contrato. Las piezas necesarias están: solo falta envolver al `ReportService` con un servidor LSP.

11.5. Soporte para métodos y clases

Hoy se analizan solo funciones a nivel de módulo. Extender el soporte a métodos requiere decidir qué hacer con `self/cls`, atributos referenciados y herencia. Es una buena oportunidad para introducir un modelo más rico de *ámbitos* de nombres.

12. Conclusiones

CDCS Mini cumple los requisitos del desafío original que le da origen y, sobre esa base, propone una arquitectura limpia que sostiene tres superficies (CLI, HTTP, Web) sin duplicar lógica. La elección de Python 3.12 con tipado estricto y la insistencia en el determinismo del reporte son decisiones que se pagan en tiempo de desarrollo pero que recompensan al revisor: el código se puede leer de afuera hacia adentro y cada decisión tiene un porqué visible.

Más allá de los entregables, el proyecto sirve como ejercicio honesto de las primeras fases de una herramienta tipo compilador. Las líneas futuras descritas son extensiones plausibles, no fantasiosas: cada una se apoya en la estructura existente y abre un camino concreto hacia una tesina de mayor profundidad. En esa medida, este documento se ofrece como punto de partida y no como cierre.

Referencias

Referencias

[1] Python Software Foundation. *ast — Abstract Syntax Trees*. Documentación oficial, Python 3.12. https://docs.python.org/3/library/ast.html.

[2] Aho, A. V., Lam, M. S., Sethi, R., Ullman, J. D. *Compilers: Principles, Techniques, and Tools*, 2.ª edición. Pearson, 2006.

[3] Meyer, B. *Applying Design by Contract*. IEEE Computer, 25(10):40–51, 1992.

[4] Ramírez, S. *FastAPI*. Documentación oficial. https://fastapi.tiangolo.com/.

[5] Vercel. *Next.js Documentation*. https://nextjs.org/docs.

[6] Vazou, N., Seidel, E. L., Jhala, R., Vytiniotis, D., Peyton Jones, S. *Refinement Types for Haskell*. ICFP, 2014.

[7] MacIver, D. R., Hatfield-Dodds, Z. *Hypothesis: a new approach to property-based testing*. Journal of Open Source Software, 4(43):1891, 2019.